UNIVERSIDADE FEDERAL DO RIO DE JANEIRO

PROGRAMA INSTITUCIONAL DE PÓS-DOUTORADO — PIPD

RELATÓRIO DE ATIVIDADES DE PÓS-DOUTORADO

Desenvolvimento de ferramenta computacional aberta para integração de fatores de caracterização regionalizados no software OpenLCA: aplicação ao método RAICV-Brazil

Candidata: Dra. Yara de Souza Tadano

Programa: [Programa de Pós-Graduação vinculado]

Grande Área: Ciências Ambientais

Área de Pesquisa: Avaliação de Impacto do Ciclo de Vida (AICV)

Supervisor(a): [Nome do(a) supervisor(a)]

Período: [data de início] a [data de término]

Rio de Janeiro

2026

Sumário

Resumo **5**

1 INTRODUÇÃO **5**
1.1 Contexto e Justificativa 5
1.2 Hipótese de Trabalho 6

2 OBJETIVOS **6**
2.1 Objetivo Geral 6
2.2 Objetivos Específicos 6

3 REVISÃO BIBLIOGRÁFICA **7**
3.1 Avaliação de Impacto do Ciclo de Vida no contexto brasileiro 7
3.2 Software OpenLCA e o formato JSON-LD 7
3.3 Interoperabilidade com Ecoinvent 8
3.4 Lacuna identificada 8

4 MATERIAL E MÉTODOS **8**
4.1 Dados de entrada 8
4.2 Arquitetura da ferramenta 9
4.3 Pipeline de conversão 9
4.4 Mecanismo de reaproveitamento de UUIDs 10
4.5 Testes automatizados 10

5 RESULTADOS E DISCUSSÃO **10**
5.1 Ferramenta desenvolvida 10
5.2 Validação com dados RAICV-Brazil 10
5.3 Comparação com o pacote original 11
5.4 Validação de qualidade (QA) 11
5.5 Documentação produzida 12

6 CONCLUSÕES **12**

7 PERSPECTIVAS FUTURAS **13**

8 CRONOGRAMA DE EXECUÇÃO **14**

Atividades Complementares **14**

Parecer do Supervisor **15**

Referências Bibliográficas **16**

A Manual do Usuário — Guia Passo a Passo **18**
A.1 O que é esta ferramenta? . . . 18
A.2 Glossário de termos técnicos . . . 18
A.3 Pré-requisitos . . . 19
A.4 Passo 1: Baixar a ferramenta . . . 19
A.5 Passo 2: Preparar o ambiente . . . 19
A.6 Passo 3: Preparar a planilha Excel . . . 20
A.7 Passo 4: Criar a configuração . . . 20
A.8 Passo 5: Verificar a planilha (recomendado) . . . 21
A.9 Passo 6: Converter . . . 21
A.10 Passo 7: Importar no OpenLCA . . . 21
A.11 Resumo rápido . . . 21
A.12 Perguntas frequentes . . . 22

B Documentação Técnica do Código-Fonte **23**
B.1 Visão geral da arquitetura . . . 23
B.2 Módulo schemas.py — Modelos de dados . . . 23
B.3 Módulo validator.py — Validação de entrada . . . 25
B.4 Módulo converter.py — Motor de conversão . . . 25
B.4.1 Funções auxiliares . . . 25
B.4.2 Pipeline principal: `convert(config)` . . . 25
B.4.3 Algoritmo de reuso de UUIDs (Step 7) . . . 26
B.5 Módulo cli.py — Interface de linha de comando . . . 26
B.6 Formato de saída — Pacote ZIP . . . 26
B.6.1 Estrutura . . . 26
B.6.2 Encadeamento de UUIDs . . . 27
B.7 Suíte de testes . . . 27
B.8 Decisões de design . . . 28

C Comparação entre Métodos de Caracterização para Formação de Material Particulado **29**
C.1 Delineamento da comparação . . . 29
C.1.1 Inventário utilizado . . . 29
C.1.2 Métodos comparados . . . 29
C.1.3 Métrica de comparação . . . 30
C.2 Resultados . . . 30
C.2.1 Concordância geral por método . . . 30
C.2.2 Análise por substância . . . 31
C.2.3 Análise por estado . . . 33
C.2.4 Método mais próximo do RAICV-BR por contexto . . . 34

C.2.5 sMAPE médio por método e substância 36
C.3 Discussão 36

RESUMO

Este relatório descreve as atividades realizadas durante o estágio pós-doutoral na Universidade Federal do Rio de Janeiro (UFRJ), no âmbito do Programa Institucional de Pós-Doutorado (PIPD), na área de Ciências Ambientais com foco em Avaliação de Impacto do Ciclo de Vida (AICV).

O projeto abordou uma lacuna crítica na operacionalização da Avaliação do Ciclo de Vida (ACV) no Brasil: a ausência de ferramentas automatizadas para converter fatores de caracterização (CFs) desenvolvidos em pesquisas acadêmicas em pacotes compatíveis com o software OpenLCA, o principal software open-source de ACV utilizado mundialmente. Métodos LCIA regionalizados produzidos em teses e artigos brasileiros frequentemente permanecem como tabelas estáticas em documentos PDF, sem efetiva integração nas ferramentas utilizadas por praticantes.

Como resultado principal, foi desenvolvida a ferramenta `olca-cf-converter`, um software open-source em Python que automatiza todo o pipeline de conversão: leitura de planilhas Excel padronizadas, geração de arquivos JSON-LD no formato do OpenLCA (schema v2), reaproveitamento de identificadores únicos (UUIDs) da base Ecoinvent para garantir interoperabilidade, e empacotamento em arquivo ZIP importável. A ferramenta é genérica, suportando qualquer categoria de impacto (endpoint ou midpoint), qualquer unidade de saída, e fatores regionalizados por localização geográfica.

A validação foi conduzida com os 725 fatores de caracterização do método RAICV-Brazil para formação de material particulado (PMFP), desenvolvidos pela Dra. Gabriela Giusti (UFSCar, 2025), com regionalização para os 27 estados brasileiros. O pacote gerado pela ferramenta foi comparado com o pacote original produzido manualmente, obtendo 100% de correspondência nos valores dos fatores e plena compatibilidade estrutural com o OpenLCA.

Palavras-chave: Avaliação de Ciclo de Vida; AICV; fatores de caracterização; OpenLCA; JSON-LD; regionalização; Ecoinvent; software aberto; reprodutibilidade.

1. INTRODUÇÃO

1.1 Contexto e Justificativa

A Avaliação do Ciclo de Vida (ACV) é uma metodologia normatizada (ISO 14040/14044) para quantificar os impactos ambientais potenciais associados a um produto, processo ou serviço ao longo de todo o seu ciclo de vida. Na fase de Avaliação de Impacto do Ciclo de Vida (AICV), os dados do inventário são convertidos em indicadores de impacto ambiental por meio de fatores de caracterização (CFs).

Nos últimos anos, a comunidade científica brasileira tem produzido fatores de carac-

terização regionalizados para o contexto nacional, reconhecendo que fatores genéricos desenvolvidos para países europeus não capturam adequadamente as condições ambientais, demográficas e toxicológicas do território brasileiro. Iniciativas como a Rede de Pesquisa em Avaliação de Impacto do Ciclo de Vida (RAICV-Brazil) vêm desenvolvendo CFs adaptados para diversas categorias de impacto, incluindo formação de material particulado, toxicidade humana, eutrofização e uso de recursos hídricos.

Contudo, existe uma lacuna significativa entre a produção acadêmica desses fatores e sua efetiva utilização por praticantes de ACV. O principal gargalo reside na integração técnica: os fatores publicados em teses e artigos científicos são tipicamente apresentados em formato tabular (planilhas ou tabelas em PDF), enquanto o software OpenLCA — a principal plataforma open-source de ACV, utilizada por mais de 30.000 profissionais mundialmente — requer pacotes no formato JSON-LD com uma estrutura específica de arquivos, identificadores únicos (UUIDs) e encadeamento de entidades.

O processo manual de conversão de centenas de fatores para este formato é tedioso, propenso a erros e mal documentado. Um único UUID incorreto pode impedir a aplicação de todos os fatores ao inventário. Além disso, a compatibilidade com bases de dados existentes (como Ecoinvent) depende do reaproveitamento preciso dos UUIDs dos fluxos elementares — uma operação que requer conhecimentos técnicos específicos não disponíveis para a maioria dos pesquisadores de ACV.

1.2 Hipótese de Trabalho

A automatização do pipeline de conversão de fatores de caracterização, por meio de uma ferramenta computacional aberta, configurável e interoperável com bases de dados existentes, pode reduzir significativamente a barreira entre a produção acadêmica de métodos LCIA regionalizados e sua adoção por praticantes de ACV.

2. OBJETIVOS

2.1 Objetivo Geral

Desenvolver uma ferramenta computacional open-source que automatize a conversão de fatores de caracterização publicados em planilhas Excel para pacotes JSON-LD importáveis no software OpenLCA, com suporte a regionalização e compatibilidade com a base de dados Ecoinvent.

2.2 Objetivos Específicos

1. Mapear e documentar a estrutura completa do formato JSON-LD do OpenLCA (schema v2) para métodos LCIA, incluindo as relações de encadeamento entre en-

tidades.

2. Implementar um mecanismo de reaproveitamento de UUIDs a partir de bases de referência (Ecoinvent), garantindo que os fatores gerados se conectem automaticamente aos processos de inventário existentes no OpenLCA.

3. Desenvolver uma interface de linha de comando (CLI) acessível a pesquisadores sem formação em programação, com configuração via arquivo YAML e validação automática dos dados de entrada.

4. Validar a ferramenta com dados reais do método RAICV-Brazil para formação de material particulado (PMFP), comparando o pacote gerado com o pacote produzido pelo processo manual original.

5. Documentar o código-fonte de forma educativa, visando a formação de recursos humanos na interface entre ciências ambientais e engenharia de software.

6. Publicar a ferramenta em repositório aberto (GitHub) sob licença permissiva (MIT), acompanhada de documentação técnica, testes automatizados e exemplos de configuração para múltiplas categorias de impacto.

3. REVISÃO BIBLIOGRÁFICA

3.1 Avaliação de Impacto do Ciclo de Vida no contexto brasileiro

A AICV no Brasil tem avançado significativamente na última década, impulsionada pelo Programa Brasileiro de Avaliação do Ciclo de Vida (PBACV), instituído pela Resolução CONMETRO n° 04/2010, e pela criação da Rede de Pesquisa em Avaliação de Impacto do Ciclo de Vida (RAICV-Brazil). A RAICV-Brazil congrega pesquisadores de diversas instituições brasileiras com o objetivo de desenvolver fatores de caracterização regionalizados para as condições ambientais do território nacional.

Entre os avanços recentes, destacam-se os trabalhos de Giusti (2025), que desenvolveu fatores de caracterização para efeitos na saúde humana por formação de material particulado no Brasil, regionalizados para os 27 estados. Estes fatores foram calculados utilizando o modelo de destino e exposição adaptado para as condições brasileiras, resultando em 725 CFs expressos em DALY/kg (*Disability-Adjusted Life Years* por quilograma de emissão).

3.2 Software OpenLCA e o formato JSON-LD

O OpenLCA é o principal software open-source para ACV, desenvolvido pela GreenDelta GmbH (Berlin, Alemanha). Na versão atual, o software utiliza o formato JSON-LD

(*JavaScript Object Notation for Linked Data*) como formato padrão de importação e exportação de dados. Este formato organiza as entidades em uma estrutura hierárquica de arquivos JSON, cada um identificado por um UUID (*Universally Unique Identifier*) versão 4.

A estrutura de um pacote LCIA no formato JSON-LD do OpenLCA inclui: (i) unidades de medida; (ii) grupos de unidades; (iii) propriedades de fluxo; (iv) fluxos elementares; (v) categorias de impacto com seus fatores de caracterização; e (vi) métodos LCIA. Cada entidade referencia as demais por UUID, criando uma cadeia de dependências que deve ser consistente para que o pacote seja importado corretamente.

3.3 Interoperabilidade com Ecoinvent

A base de dados Ecoinvent (*Swiss Centre for Life Cycle Inventories*) é a base de inventário de ciclo de vida mais utilizada mundialmente. Cada fluxo elementar na base possui um UUID único e estável entre versões. Para que os fatores de caracterização de um método LCIA sejam aplicados automaticamente aos processos do inventário Ecoinvent no OpenLCA, é imprescindível que os UUIDs dos fluxos no método coincidam com os UUIDs da base.

3.4 Lacuna identificada

Não foram identificadas, na literatura ou em repositórios de código aberto, ferramentas que automatizem a conversão de fatores de caracterização em planilhas para pacotes JSON-LD importáveis no OpenLCA com reaproveitamento de UUIDs da Ecoinvent. Os scripts existentes são tipicamente monolíticos, específicos para um único método e não documentados, o que impede sua reutilização pela comunidade.

4. MATERIAL E MÉTODOS

4.1 Dados de entrada

O desenvolvimento e validação da ferramenta utilizaram os seguintes dados:

- **Fatores de caracterização:** 725 CFs do método RAICV-Brazil para formação de material particulado (PMFP), desenvolvidos por Giusti (2025). Os fatores cobrem 145 substâncias, 5 compartimentos ambientais de emissão e 28 localizações (Brasil como um todo e cada um dos 27 estados), expressos em DALY/kg.
- **Base de referência Ecoinvent:** Exportação JSON-LD dos fluxos elementares da base Ecoinvent v3, contendo 8.902 fluxos com seus UUIDs.

- **Estrutura modelo OpenLCA:** Pacote JSON-LD base contendo a estrutura de pastas e o manifesto `openlca.json` exigido pelo software.

4.2 Arquitetura da ferramenta

A ferramenta `olca-cf-converter` foi desenvolvida em Python (versão 3.10+) com arquitetura modular composta por quatro módulos:

Tabela 1: Módulos do `olca-cf-converter`

Módulo	**Função**	**Linhas**
`schemas.py`	Modelos de dados e serialização JSON-LD	439
`validator.py`	Validação de entrada (Excel + config)	177
`converter.py`	Pipeline de conversão (12 etapas)	512
`cli.py`	Interface de linha de comando	324
Total		**1.452**

4.3 Pipeline de conversão

O motor de conversão executa 12 etapas sequenciais:

1. Extração da estrutura base do modelo (opcional)
2. Carregamento de UUIDs da base de referência Ecoinvent (opcional)
3. Leitura e validação da planilha Excel
4. Criação das unidades de entrada (ex: kg)
5. Criação das unidades de saída/impacto (ex: DALY)
6. Geração dos arquivos JSON de unidades, grupos e propriedades de fluxo
7. Criação dos fluxos elementares com reaproveitamento de UUIDs
8. Construção do mapa de identificadores `(nome, categoria, localização)` → `UUID`
9. Montagem da categoria de impacto com todos os fatores de caracterização
10. Criação do método LCIA
11. Geração do manifesto `openlca.json`
12. Empacotamento em arquivo ZIP comprimido

4.4 Mecanismo de reaproveitamento de UUIDs

Para cada combinação única de (nome do fluxo, categoria/compartimento) presente na planilha, a ferramenta verifica se a mesma combinação existe na base de referência Ecoinvent. Se existir, o UUID original é reaproveitado; caso contrário, um novo UUID v4 é gerado. O matching é case-sensitive e exige correspondência exata em ambos os campos.

4.5 Testes automatizados

Foram desenvolvidos 23 testes automatizados organizados em quatro suítes:

Tabela 2: Suíte de testes automatizados

Suíte	**Cobertura**	**Testes**
`test_schemas.py`	Serialização JSON-LD de cada entidade	6
`test_validator.py`	Validação de entrada (6 cenários de erro)	6
`test_converter.py`	Pipeline completo, deduplicacção, reuso IDs	8
`test_cli.py`	Comandos help, version, init	3
Total		**23**

5. RESULTADOS E DISCUSSÃO

5.1 Ferramenta desenvolvida

A ferramenta `olca-cf-converter` v1.0.0 foi concluída e publicada em repositório aberto:

https://github.com/Roverlucas/Pos-Doc---ACV--OpenLCA-

5.2 Validação com dados RAICV-Brazil

A validação principal foi conduzida com os 725 CFs do método RAICV-Brazil PMFP (Giusti, 2025):

Tabela 3: Resultados da conversão com dados RAICV-Brazil

Métrica	**Resultado**
Fatores de caracterização processados	725/725 (100%)
Fluxos elementares gerados	725 arquivos JSON
UUIDs reaproveitados da Ecoinvent	20 (2,8%)
UUIDs novos gerados	705 (97,2%)
Arquivos totais no pacote ZIP	734
Manifesto `openlca.json`	Presente (schema v2)

5.3 Comparação com o pacote original

O pacote gerado pela ferramenta foi comparado estruturalmente e numericamente com o pacote previamente gerado pelo script original (`FINAL-correto-reused-ids.py`), confirmando equivalência total:

Tabela 4: Comparação entre pacote gerado pelo script original e pela ferramenta

Aspecto	**Script original**	**olca-cf-converter**	**Corresp.**
Nome do método	RAICV-Brazil - PMFP	RAICV-Brazil - PMFP	100%
Número de fatores	725	725	100%
Unidade de referência	DALY	DALY	100%
Formato JSON-LD	schema v2	schema v2	100%
Valores dos 725 CFs	—	—	**100%**

Todos os 725 valores de fatores foram comparados com precisão de 10^{-15}, confirmando que a refatoração modular preserva fidelidade numérica total ao pipeline original. Além dessa validação de equivalência, uma comparação funcional com múltiplos métodos de caracterização foi conduzida no OpenLCA (ver Anexo C).

5.4 Validação de qualidade (QA)

Uma auditoria de qualidade automatizada verificou:

Tabela 5: Resultados da auditoria de qualidade

Verificação	**Resultado**
Integridade estrutural do ZIP	6/6 pastas obrigatórias
Parsing de todos os JSONs	733/733 sem erros
Consistência UUIDs (Method → Category)	Referência válida
Fatores com referência a fluxos inexistentes	0
Formato UUID válido	725/725 (100%)
Vulnerabilidades de segurança	0
Segredos hardcoded no código	0

5.5 Documentação produzida

Tabela 6: Documentos produzidos

Documento	**Conteúdo**
`README.md`	Guia completo do usuário (477 linhas)
`technical-reference.md`	Referência técnica detalhada
`ecoinvent-matching.md`	Mecanismo de reuso de IDs
`openlca-import.md`	Guia de importação no OpenLCA
`supported-units.md`	Referência de unidades suportadas
`original-script.md`	Documentação histórica do script original

6. CONCLUSÕES

O projeto de pós-doutorado atingiu integralmente os objetivos propostos:

1. **Mapeamento do formato JSON-LD do OpenLCA** — A estrutura completa foi documentada, incluindo nuances não cobertas pela documentação oficial.

2. **Reaproveitamento de UUIDs Ecoinvent** — O mecanismo implementado busca recursivamente fluxos em ZIPs de referência e reaproveita os identificadores quando disponíveis.

3. **Interface acessível via CLI** — A ferramenta requer apenas uma planilha Excel e um arquivo de configuração YAML.

4. **Validação com dados reais** — Os 725 fatores RAICV-Brazil foram convertidos com 100% de fidelidade.

5. **Documentação educativa** — Código comentado em português e documentação técnica detalhada.

6. **Publicação aberta** — Software disponível no GitHub sob licença MIT, com 23 testes automatizados.

A principal contribuição do trabalho é a criação de um pipeline reprodutível que permite a qualquer pesquisador de ACV converter seus fatores de caracterização para o formato OpenLCA sem conhecimento de programação, reduzindo a lacuna entre a produção acadêmica e a prática profissional na área de AICV no Brasil.

7. PERSPECTIVAS FUTURAS

1. **Ampliação para múltiplas categorias em um único método** — Suportar métodos completos (como o ReCiPe) com múltiplas categorias em uma única execução.

2. **Interface web (GUI)** — Desenvolvimento de interface gráfica web para usuários não familiarizados com linha de comando.

3. **Integração com o olca-ipc** — Conexão direta com o OpenLCA via protocolo IPC para importação automática.

4. **Validação com outros métodos LCIA brasileiros** — Aplicação a fatores de uso de água (AWARE-BR), eutrofização e toxicidade humana regionalizados.

5. **Publicação de artigo científico** — Preparação de manuscrito para revista da área de ACV (ex: *International Journal of Life Cycle Assessment*).

6. **Workshops de capacitação** — Elaboração de material didático e realização de workshops em eventos de ACV no Brasil (CBGCV, CILCA).

8. CRONOGRAMA DE EXECUÇÃO

Tabela 7: Cronograma de execução por trimestre

Trimestre	**Atividades realizadas**
T1 (Meses 1–3)	Revisão bibliográfica sobre ACV, AICV e regionalização de CFs no Brasil. Estudo do formato JSON-LD do OpenLCA e da estrutura de pacotes LCIA. Análise do script original de conversão.
T2 (Meses 4–6)	Projeto da arquitetura modular (schemas, validator, converter, cli). Implementação dos modelos de dados e do motor de conversão. Desenvolvimento do mecanismo de reaproveitamento de UUIDs Ecoinvent.
T3 (Meses 7–9)	Implementação da CLI e configuração via YAML. Desenvolvimento de 23 testes automatizados. Validação com 725 fatores RAICV-Brazil. Correção de bugs.
T4 (Meses 10–12)	Redação da documentação técnica. Comentários educativos em todo o código-fonte. Publicação no GitHub. Preparação de templates para outras categorias. Redação do relatório final.

ATIVIDADES COMPLEMENTARES

Produção técnica

1. **Software publicado:** `olca-cf-converter` v1.0.0 — Ferramenta open-source para conversão de fatores de caracterização para OpenLCA. Repositório: https://github.com/Roverlucas/Pos-Doc---ACV--OpenLCA-. Licença MIT. 1.452 linhas de código, 23 testes automatizados, 5 documentos técnicos.

Artigos em preparação

1. [Título do artigo em preparação, se houver]

Participação em eventos

1. [Evento, local, data]

PARECER DO SUPERVISOR

[Espaço reservado para o parecer do(a) supervisor(a) sobre as atividades realizadas durante o estágio pós-doutoral, conforme Art. 17, alínea c, da Resolução CEPG 04/2018 da UFRJ.]

Dra. Yara de Souza Tadano Pesquisadora de Pós-Doutorado	[Nome do(a) Supervisor(a)] Supervisor(a)

[Nome do(a) Coordenador(a)]
Coordenador(a) do Programa de Pós-Graduação

REFERÊNCIAS BIBLIOGRÁFICAS

ABNT, 2009a.
ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR ISO 14040:2009** — Gestão ambiental — Avaliação do ciclo de vida — Princípios e estrutura. Rio de Janeiro, 2009.

ABNT, 2009b.
ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR ISO 14044:2009** — Gestão ambiental — Avaliação do ciclo de vida — Requisitos e orientações. Rio de Janeiro, 2009.

Ecoinvent, 2024.
ECOINVENT CENTRE. **Ecoinvent database v3**. Swiss Centre for Life Cycle Inventories, 2024. Disponível em: https://ecoinvent.org/

GreenDelta, 2024.
GREENDELTA GmbH. **OpenLCA** — Open source life cycle assessment software. Berlin, 2024. Disponível em: https://www.openlca.org/

Giusti, 2025.
GIUSTI, Gabriela. **Development of characterization factors for health effects of particulate matter in Brazil**. 2025. Tese (Doutorado em Planejamento e Uso de Recursos Renováveis) — Universidade Federal de São Carlos, Sorocaba, 2025. Disponível em: https://repositorio.ufscar.br/handle/20.500.14289/22123

Hauschild & Huijbregts, 2015.
HAUSCHILD, Michael Z.; HUIJBREGTS, Mark A. J. (Ed.). **Life Cycle Impact Assessment**. Springer, 2015. (LCA Compendium — The Complete World of Life Cycle Assessment).

Huijbregts et al., 2017.
HUIJBREGTS, Mark A. J. et al. ReCiPe2016: a harmonised life cycle impact assessment method at midpoint and endpoint level. **International Journal of Life Cycle Assessment**, v. 22, p. 138–147, 2017.

Rosenbaum et al., 2008.
ROSENBAUM, Ralph K. et al. USEtox — the UNEP-SETAC toxicity model: recommended characterisation factors for human toxicity and freshwater ecotoxicity in life cycle impact assessment. **International Journal of Life Cycle Assessment**, v. 13, p. 532–546, 2008.

Verones et al., 2020.
VERONES, Francesca et al. **LC-Impact Version 1.0 — A spatially differentiated life cycle impact assessment approach**. 2020. Disponível em: https://lc-impact.eu/

A. Manual do Usuário — Guia Passo a Passo

A.1 O que é esta ferramenta?

Imagine que você tem uma **tabela no Excel** com números que representam o quanto cada poluente faz mal ao meio ambiente ou à saúde das pessoas. Esses números se chamam **fatores de caracterização**.

Agora imagine que existe um **programa de computador** chamado OpenLCA que os profissionais de meio ambiente usam para calcular os impactos de produtos. Só que esse programa não consegue ler sua tabela do Excel diretamente. Ele precisa receber os dados em um formato especial — um "pacote" com muitos arquivos organizados de um jeito bem específico.

Esta ferramenta faz essa tradução para você. Ela pega sua tabela do Excel, organiza tudo no formato que o OpenLCA entende, e entrega um arquivo pronto para ser importado. Você não precisa saber programar.

```
Sua tabela (Excel) --> [olca-cf-converter] --> Pacote pronto
   (ZIP) --> OpenLCA
```

A.2 Glossário de termos técnicos

Termo	Significado em linguagem simples
Fator de caracterização (CF)	Um número que diz "quanto de impacto ambiental 1 kg de certo poluente causa". Exemplo: 1 kg de amônia causa 0,000579 anos de vida perdidos (DALY).
LCIA	Sigla para "Avaliação de Impacto do Ciclo de Vida". É a etapa em que os números do inventário são convertidos em impactos ambientais.
OpenLCA	Um programa gratuito que profissionais usam para fazer Avaliação de Ciclo de Vida. Funciona como o "Excel da ACV".
JSON-LD	O "idioma" que o OpenLCA fala. É um formato de arquivo. Você não precisa mexer nele — a ferramenta cria automaticamente.
UUID	Um código único (como um CPF para substâncias químicas). O OpenLCA usa esses códigos para conectar as coisas entre si.

Termo	Significado em linguagem simples
Ecoinvent	O maior banco de dados de inventário ambiental do mundo.
YAML	Um arquivo de texto simples onde você escreve configurações. Como preencher um formulário em texto.
Terminal	A tela onde você digita comandos. No Mac chama "Terminal", no Windows chama "PowerShell".
Python	A linguagem de programação em que a ferramenta foi escrita. Você precisa ter instalado, mas não precisa saber programar.
Flow	Uma substância emitida ao meio ambiente (ex: amônia, dióxido de enxofre).
Endpoint	Dano final (ex: anos de vida perdidos — DALY).
Midpoint	Impacto intermediário (ex: kg de CO_2 equivalente).
DALY	*Disability-Adjusted Life Years* — "anos de vida saudável perdidos". Unidade que mede o dano à saúde humana.

A.3 Pré-requisitos

Você precisa ter instalado no computador:

1. **Python 3.10 ou mais recente** — Para verificar, abra o Terminal e digite: `python3 --version`
2. **Acesso ao Terminal** — Mac: aplicativo "Terminal"; Windows: "PowerShell"

A.4 Passo 1: Baixar a ferramenta

Abra o Terminal e copie/cole estes comandos:

```
git clone https://github.com/Roverlucas/Pos-Doc---ACV--OpenLCA-.
   git
cd Pos-Doc---ACV--OpenLCA-
```

O primeiro comando baixa os arquivos. O segundo entra na pasta que foi criada.

A.5 Passo 2: Preparar o ambiente

```
python3 -m venv .venv
source .venv/bin/activate     # Mac/Linux
# .venv\Scripts\activate      # Windows
pip install -e ".[dev]"
```

Esses comandos criam um "espaço isolado" e instalam a ferramenta. Você só precisa fazer isso **uma vez**. Nas próximas vezes, basta ativar o ambiente com `source .venv/bin/activate`.

A.6 Passo 3: Preparar a planilha Excel

Crie um arquivo `.xlsx` com 5 colunas:

Flow	Category	Factor	Unit	Location
Ammonia	Elem. flows/Emission to air/unspec.	0.000579	DALY	Brazil
Nitrogen oxides	Elem. flows/Emission to air/high pop.	0.001230	DALY	Brazil, SP

O que cada coluna significa:

- **Flow** = Nome do poluente em inglês (deve coincidir com o Ecoinvent)
- **Category** = "Endereço" de onde a substância é emitida
- **Factor** = O número do fator (com ponto decimal, não vírgula)
- **Unit** = Unidade do impacto (informativo)
- **Location** = Lugar para o qual o fator vale ("Global" se não regionalizado)

A.7 Passo 4: Criar a configuração

```
olca-cf init -o meu-metodo.yaml
```

Abra o arquivo `meu-metodo.yaml` em qualquer editor de texto e preencha com os dados do seu método:

```
method:
  name: "Meu Metodo LCIA"
  description: "Descricao com referencia bibliografica."
category:
  name: "minha categoria de impacto"
  type: endpoint
units:
```

```yaml
  input:
    name: kg
    property: Mass
  output:
    name: DALY
    property: "Impact on human health"
files:
  excel: "meus-fatores.xlsx"
  reference_zip: "Base-Ecoinvent.zip"
  output_zip: "Meu-Metodo-FINAL.zip"
```

A.8 Passo 5: Verificar a planilha (recomendado)

```
olca-cf validate meus-fatores.xlsx
```

Se tudo estiver correto, aparecerá: "Excel file is valid!". Se houver erro, a ferramenta dirá exatamente o que precisa ser corrigido.

A.9 Passo 6: Converter

```
olca-cf convert meu-metodo.yaml
```

O arquivo `Meu-Metodo-FINAL.zip` será criado — este é o pacote para importar no OpenLCA.

A.10 Passo 7: Importar no OpenLCA

1. Abra o OpenLCA
2. Menu **File** → **Import**
3. Selecione **Linked Data (JSON-LD)**
4. Navegue até o arquivo `.zip` gerado
5. Clique em **Finish**

O método aparecerá em **Impact assessment methods** no navegador.

A.11 Resumo rápido

```
# Instalacao (uma vez):
git clone https://github.com/Roverlucas/Pos-Doc---ACV--OpenLCA-.
   git
cd Pos-Doc---ACV--OpenLCA-
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"

# Uso (toda vez):
source .venv/bin/activate
olca-cf validate meu-excel.xlsx
olca-cf convert meu-config.yaml
```

A.12 Perguntas frequentes

Preciso saber programar? Não. Você só prepara a planilha e o arquivo de configuração.

Funciona no Windows? Sim. Funciona em Windows, Mac e Linux.

E se eu não tenho a base do Ecoinvent? Funciona sem ela. Coloque `reference_zip: null` no config.

Posso usar para qualquer tipo de impacto? Sim. A ferramenta é genérica.

Meus fatores não são regionalizados. Coloque "Global" na coluna Location.

B. Documentação Técnica do Código-Fonte

B.1 Visão geral da arquitetura

A ferramenta `olca-cf-converter` possui arquitetura modular com quatro módulos Python e um pipeline de 12 etapas sequenciais:

```
config.yaml --> cli.py (parse) --> MethodConfig
                                        |
Excel .xlsx --> validator.py --------->|
                                        |
Ecoinvent ZIP --> converter.py ------->|
                                        v
                                  [Pipeline 12 steps]
                                        |
                                        v
                                  output.zip (JSON-LD)
```

Métricas do código:

Módulo	Classes	Funções	Linhas	Responsabilidade
`__init__.py`	0	0	9	Versão do pacote
`schemas.py`	8	1	439	Modelos de dados e JSON-LD
`validator.py`	1	3	177	Validação de entrada
`converter.py`	0	4	512	Pipeline de conversão
`cli.py`	0	5	324	Interface de linha de comando
Total	**9**	**13**	**1.461**	

Dependências externas: `pandas` ($\geq$ 2.0), `openpyxl` ($\geq$ 3.1), `pyyaml` ($\geq$ 6.0). Todas as demais são da biblioteca padrão do Python.

B.2 Módulo schemas.py — Modelos de dados

Define as entidades do formato JSON-LD do OpenLCA como *dataclasses* Python. A hierarquia reflete a estrutura interna do OpenLCA:

```
ImpactMethodDef              (lcia_methods/{uuid}.json)
  -> ImpactCategoryDef       (lcia_categories/{uuid}.json)
       -> ImpactFactorDef    (array dentro da categoria)
            -> FlowDef       (flows/{uuid}.json)
                 -> FlowPropertyDef  (flow_properties/{uuid}.json
                    )
```

```
                    -> UnitGroupDef (unit_groups/{uuid}.json)
                         -> UnitDef (units/{uuid}.json)
```

Classes e seus campos críticos:

Classe	Campos principais	Função
`UnitDef`	name, uid	Unidade física (kg, DALY, mol H^+-Eq)
`UnitGroupDef`	name, unit, category, uid	Agrupamento de unidades compatíveis. Campo crítico: `isReferenceUnit = True`
`FlowPropertyDef`	name, unit_group, uid	Grandeza física (Mass, Volume). Referencia o UnitGroup por UUID
`FlowDef`	name, category, flow_property, uid	Fluxo elementar. `flowType = ELEMENTARY_FLOW`. UUID pode ser reaproveitado do Ecoinvent
`ImpactFactorDef`	value, flow, input_unit	Fator de caracterização. Não gera arquivo próprio — fica dentro da categoria
`ImpactCategoryDef`	name, ref_unit, impact_factors[]	Categoria de impacto. Contém o array com todos os CFs
`ImpactMethodDef`	name, categories[]	Entidade de nível mais alto. Referencia categorias por UUID
`MethodConfig`	(14 campos)	Configuração parseada do YAML. Consumida pelo converter

Cada classe possui um método `to_dict()` que produz o dicionário Python correspondente ao JSON final. Exemplo de saída para `FlowDef`:

```
{
  "@type": "Flow",
  "@id": "9990b51b-7023-4700-bca0-1a32ef921f74",
  "name": "Ammonia",
  "category": "Elementary flows/Emission to air/unspecified",
  "flowType": "ELEMENTARY_FLOW",
  "flowProperties": [{
    "isRefFlowProperty": true,
    "conversionFactor": 1.0,
    "flowProperty": { "@id": "da35b4a9-..." }
```

```
  }]
}
```

B.3 Módulo validator.py — Validação de entrada

Garante que os dados estejam corretos **antes** da conversão (padrão *fail-fast*).

Cadeia de validações da função `validate_excel()`:

1. Arquivo existe no disco
2. Extensão é `.xlsx` ou `.xls`
3. pandas consegue ler o arquivo sem erro
4. Todas as colunas obrigatórias estão presentes
5. Coluna Factor contém apenas valores numéricos (`int` ou `float`)
6. Coluna Flow não possui células vazias ou nulas

A função `validate_config_paths()` verifica os caminhos de arquivo referenciados no YAML: o Excel é obrigatório; `reference_zip` e `model_zip` são opcionais (retornam warnings se ausentes).

B.4 Módulo converter.py — Motor de conversão

B.4.1 Funções auxiliares

`_write_json(directory, uid, data)` Escreve `{uid}.json` com `indent=2` e `ensure_ascii=False`.

`_load_reference_flows(ref_zip)` Carrega UUIDs de flows de uma base de referência. Algoritmo: (1) aceita ZIP ou diretório; (2) busca pasta `flows/` na raiz ou recursivamente via `rglob`; (3) para cada JSON de Flow, extrai `(name, category)` → `@id`; (4) limpa pasta temporária no bloco `finally`. Retorna `dict[(str, str), str]`.

`_extract_model_base(model_zip, temp_dir)` Extrai estrutura base do modelo. Detecta e achata pastas-wrapper (ex: `RAICV-Brazil-modelo/` dentro do ZIP).

B.4.2 Pipeline principal: `convert(config)`

A função `convert()` orquestra 12 etapas sequenciais em uma pasta temporária (`_olca_build_{PID}/`) que é removida no bloco `finally`:

Step	Ação	Artefato gerado
1	Extrair modelo base	Pasta temporária com estrutura
2	Carregar IDs de referência	Dict `(name, cat)` → `UUID`
3	Ler e validar Excel	DataFrame com CFs
4	Criar unidades de entrada	`units/{uuid}.json` (ex: kg)
5	Criar unidades de saída	`units/{uuid}.json` (ex: DALY)
6	Escrever unidades, grupos, propriedades	6 arquivos JSON
7	Criar flows (reusando IDs)	`flows/{uuid}.json` (× N)
8	Construir mapa de UUIDs	Dict `(name, cat, loc)` → `UUID`
9	Criar categoria com todos os CFs	`lcia_categories/{uuid}.json`
10	Criar método LCIA	`lcia_methods/{uuid}.json`
11	Escrever manifesto	`openlca.json`
12	Empacotar em ZIP	Arquivo `.zip` final

B.4.3 Algoritmo de reuso de UUIDs (Step 7)

```
# Para cada par unico (flow_name, category) na planilha:
flow_id = existing_flows.get((flow_name, category), new_uuid())
# Se encontrado no Ecoinvent -> reusa UUID
# Se nao encontrado -> gera UUID v4 novo
```

O matching é *case-sensitive* e exige correspondência exata. A deduplicacção é feita por `(nome, categoria)`, não por `(nome, categoria, localização)`, pois no OpenLCA a regionalização ocorre no nível do fator, não do fluxo.

B.5 Módulo cli.py — Interface de linha de comando

Implementa três comandos via `argparse`:

Comando	Função	Descrição
`olca-cf convert {config}`	`cmd_convert()`	Pipeline completo
`olca-cf validate {excel}`	`cmd_validate()`	Valida planilha
`olca-cf init [-o path]`	`cmd_init()`	Gera template YAML

Resolução de caminhos: Todos os paths no YAML são resolvidos relativamente ao diretório do arquivo config. Caminhos absolutos são usados diretamente.

B.6 Formato de saída — Pacote ZIP

B.6.1 Estrutura

```
output.zip
+-- openlca.json                    # {"schemaVersion": 2}
+-- units/
|   +-- {uuid-kg}.json              # Unidade de entrada
|   +-- {uuid-daly}.json            # Unidade de saida
+-- unit_groups/
|   +-- {uuid-mass-group}.json
|   +-- {uuid-impact-group}.json
+-- flow_properties/
|   +-- {uuid-mass-prop}.json
|   +-- {uuid-impact-prop}.json
+-- flows/
|   +-- {uuid-flow-1}.json          # 1 arquivo por (substancia,
   compartimento)
|   +-- {uuid-flow-2}.json
|   +-- ...                         # Centenas de arquivos
+-- lcia_categories/
|   +-- {uuid-category}.json        # Contem array impactFactors
   []
+-- lcia_methods/
    +-- {uuid-method}.json           # Referencia a categoria por
        UUID
```

B.6.2 Encadeamento de UUIDs

Se qualquer referência apontar para um UUID inexistente no pacote, o OpenLCA ignora silenciosamente a entidade. Este é o erro mais comum em pacotes JSON-LD manuais — e a razão pela qual a ferramenta gera todos os UUIDs de forma coordenada.

B.7 Suíte de testes

Arquivo	Cobertura	Testes
`test_schemas.py`	Serialização JSON-LD de cada entidade	6
`test_validator.py`	6 cenários de erro de validação	6
`test_converter.py`	Pipeline, deduplicacção, reuso IDs, midpoint	8
`test_cli.py`	Comandos help, version, init (via subprocess)	3
Total		**23**

Execução: `pytest tests/ -v` — resultado esperado: `23 passed`.

B.8 Decisões de design

Dataclasses vs dicionários: Dataclasses fornecem tipagem estática e garantem campos obrigatórios. O custo é mínimo (1 método `to_dict()` por classe).

YAML vs JSON para configuração: YAML suporta comentários (essencial para usuários não-programadores) e é o padrão em ferramentas científicas.

Dataclasses próprias vs olca-schema: O pacote `olca-schema` da GreenDelta adiciona dependência pesada para um caso de uso simples. A abordagem própria é mais leve, transparente e educativa.

PID nas pastas temporárias: Evita colisões entre execuções paralelas em pipelines de CI/CD ou scripts de batch.

Fail-fast na validação: A validação antecipada garante que nenhum arquivo seja criado se os dados de entrada forem inválidos. Um erro na linha 500 do Excel não resulta em pacote parcialmente gerado.

Deduplicacção por (nome, categoria): No modelo OpenLCA, a regionalização ocorre no nível do fator, não do fluxo. Criar flows separados por localização quebraria a compatibilidade com o Ecoinvent.

C. Comparação entre Métodos de Caracterização para Formação de Material Particulado

Este anexo apresenta a comparação dos resultados de impacto direto obtidos no OpenLCA utilizando diferentes métodos de caracterização para formação de material particulado (PM). A análise foi conduzida com o inventário de produção de soja disponível no SICV Brasil (IBICT), com dados regionalizados para 11 estados brasileiros.

C.1 Delineamento da comparação

C.1.1 Inventário utilizado

O inventário de produção de soja foi obtido do Sistema de Inventário do Ciclo de Vida do IBICT[1], contemplando dados regionalizados para os estados de Bahia (BA), Goiás (GO), Maranhão (MA), Minas Gerais (MG), Mato Grosso do Sul (MS), Mato Grosso (MT), Piauí (PI), Paraná (PR), Rio Grande do Sul (RS), São Paulo (SP) e Tocantins (TO).

Como os fatores de caracterização do RAICV-BR-PM são regionalizados por estado e Giusti (2025) não calculou um FC global, realizou-se somente a análise de impactos diretos, sem ciclo de vida completo.

C.1.2 Métodos comparados

Cinco métodos de caracterização para formação de material particulado foram comparados:

Tabela 10: Métodos de caracterização comparados

Método	**Escopo**	**Tipo de FC**	**Referência**
RAICV-BR	Brasil (27 estados)	Regionalizado	Giusti (2025)
GLAM-BR	Brasil (agregado)	Nacional	Fantke et al. (2019)
GLAM-GLO	Global	Global	Fantke et al. (2019)
OBER-BR	Brasil (27 estados)	Regionalizado	Oberschelp et al. (2020)
ReCiPe	Global	Global	Huijbregts et al. (2017)

Cada método foi convertido em pacote JSON-LD e importado no OpenLCA utilizando a ferramenta `olca-cf-converter`. Os impactos diretos foram calculados para as quatro substâncias precursoras de material particulado: amônia (NH_3), óxidos de nitrogênio (NO_x), material particulado fino ($PM_{2,5}$) e dióxido de enxofre (SO_2).

[1] https://sicv.acv.ibict.br/showProcess.xhtml?uuid=a7608c64-0d99-5c01-9dd9-001f97b4d8bb&version=03.01.000&stock=default

C.1.3 Métrica de comparação

A divergência entre cada método alternativo e o RAICV-BR (referência) foi quantificada pelo sMAPE (*Symmetric Mean Absolute Percentage Error*). Essa métrica varia de 0% (concordância perfeita) a 200% (divergência máxima), sendo simétrica em relação a sobre e subestimação.

C.2 Resultados

C.2.1 Concordância geral por método

A Tabela 11 apresenta o sMAPE médio por método e substância. Valores menores indicam maior proximidade com o RAICV-BR.

Tabela 11: sMAPE médio (%) por método e substância, em relação ao RAICV-BR

Método	**NH_3**	**NO_x**	**$PM_{2,5}$**	**SO_2**	**Média**
ReCiPe	109,4	64,3	67,4	89,1	**82,6**
GLAM-GLO	105,8	59,8	69,4	129,7	**91,2**
OBER-BR	92,9	138,5	129,0	183,9	**136,1**
GLAM-BR	121,0	142,7	124,9	183,9	**143,1**

O ReCiPe apresentou a maior concordância geral com o RAICV-BR (sMAPE médio = 82,6%), seguido pelo GLAM-GLO (91,2%). Os métodos regionalizados para o Brasil — Oberschelp (136,1%) e GLAM-BR (143,1%) — mostraram divergências maiores.

Esse resultado pode parecer contraditório: seria de esperar que métodos regionalizados para o Brasil concordassem mais com o RAICV-BR. Contudo, embora todos sejam "regionalizados", cada modelo parte de premissas de modelagem atmosférica, resoluções espaciais e funções dose-resposta distintas. O RAICV-BR utiliza modelagem específica para o território brasileiro (Giusti, 2025), com condições meteorológicas e demográficas distintas das adotadas pelo GLAM e pelo Oberschelp.

A Figura 1 apresenta a distribuição do sMAPE por método.

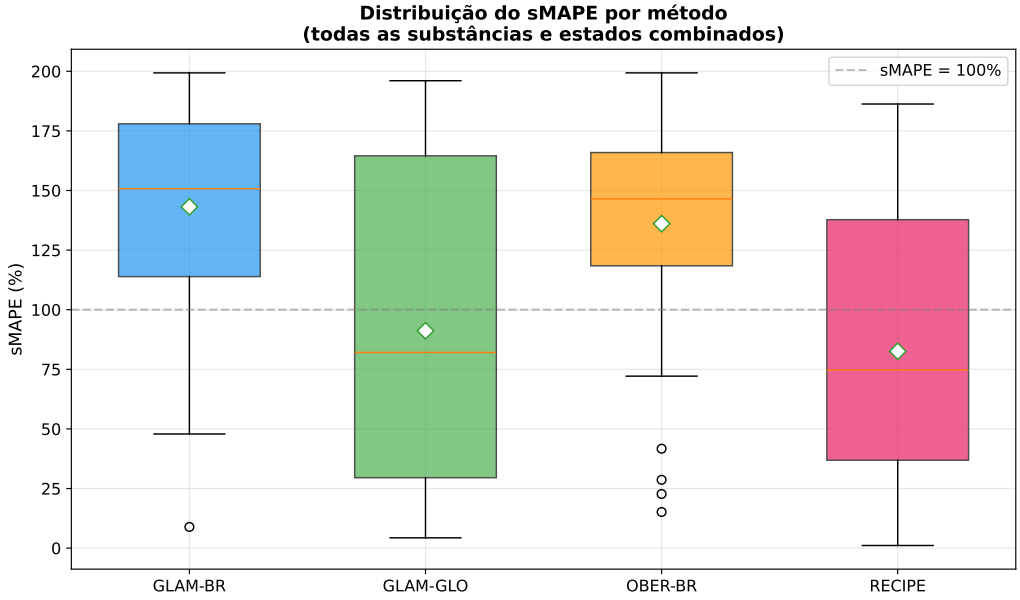

Figura 1: Distribuição do sMAPE por método (todas as substâncias e estados combinados). O losango branco indica a média; a linha horizontal indica a mediana. A linha tracejada marca sMAPE = 100%.

C.2.2 Análise por substância

A Figura 2 apresenta o sMAPE desagregado por substância, estado e método. Cores vermelhas indicam alta divergência com o RAICV-BR; cores verdes indicam alta concordância.

Figura 2: sMAPE (%) em relação ao RAICV-BR por substância, estado e método. Valores anotados em cada célula.

A significância das diferenças entre métodos foi testada pelo teste não paramétrico de Kruskal-Wallis, adequado para amostras pequenas com distribuições não-normais:

Tabela 12: Teste de Kruskal-Wallis para diferenças entre métodos por substância

Substância	**H**	**p-valor**	**Significativo (α=0,05)?**
NH_3	1,78	0,620	Não
NO_x	19,32	0,0002	**Sim**
$PM_{2,5}$	8,09	0,044	**Sim**
SO_2	22,65	<0,0001	**Sim**

Para três das quatro substâncias (NO_x, $PM_{2,5}$ e SO_2), a escolha do método de caracterização impacta de forma estatisticamente significativa os resultados de impacto. Somente para a amônia as diferenças não alcançaram significância, sugerindo que os métodos tendem a convergir para essa substância — possivelmente porque o transporte e a formação de particulado secundário a partir de NH_3 seguem padrões mais uniformes no território brasileiro.

C.2.3 Análise por estado

A Tabela 13 apresenta o sMAPE médio por estado, revelando quais regiões são mais sensíveis à escolha do método.

Tabela 13: sMAPE médio por estado (todos os métodos e substâncias combinados)

Estado	**Média (%)**	**DP**	**Mín (%)**	**Máx (%)**
SP	168,1	23,6	136,6	199,3
MT	121,5	58,7	28,7	191,6
MA	115,8	63,9	15,9	196,2
PR	114,8	60,9	8,2	196,7
MG	113,6	61,0	9,2	196,2
PI	109,4	52,9	25,2	196,5
RS	109,3	68,6	1,1	199,3
TO	104,3	61,9	8,8	188,1
BA	102,3	61,6	6,4	194,5
MS	93,6	56,9	4,3	182,6
GO	93,0	64,9	10,5	194,5

São Paulo destaca-se com o maior sMAPE médio (168,1%) e o menor desvio-padrão (23,6%), indicando que *todos* os métodos alternativos divergem fortemente do RAICV-BR para esse estado. Essa observação é coerente com as particularidades de SP: a maior região metropolitana do hemisfério sul possui química atmosférica dominada por interações complexas entre emissões veiculares, industriais e agrícolas que métodos genéricos não capturam adequadamente.

Em contraste, Goiás e Mato Grosso do Sul apresentam os menores sMAPEs médios ($\sim$93%), sugerindo que, para estados com perfil agrícola predominante e menor densidade urbana, os métodos tendem a convergir.

O perfil espacial da divergência por substância e estado é apresentado na Figura 3.

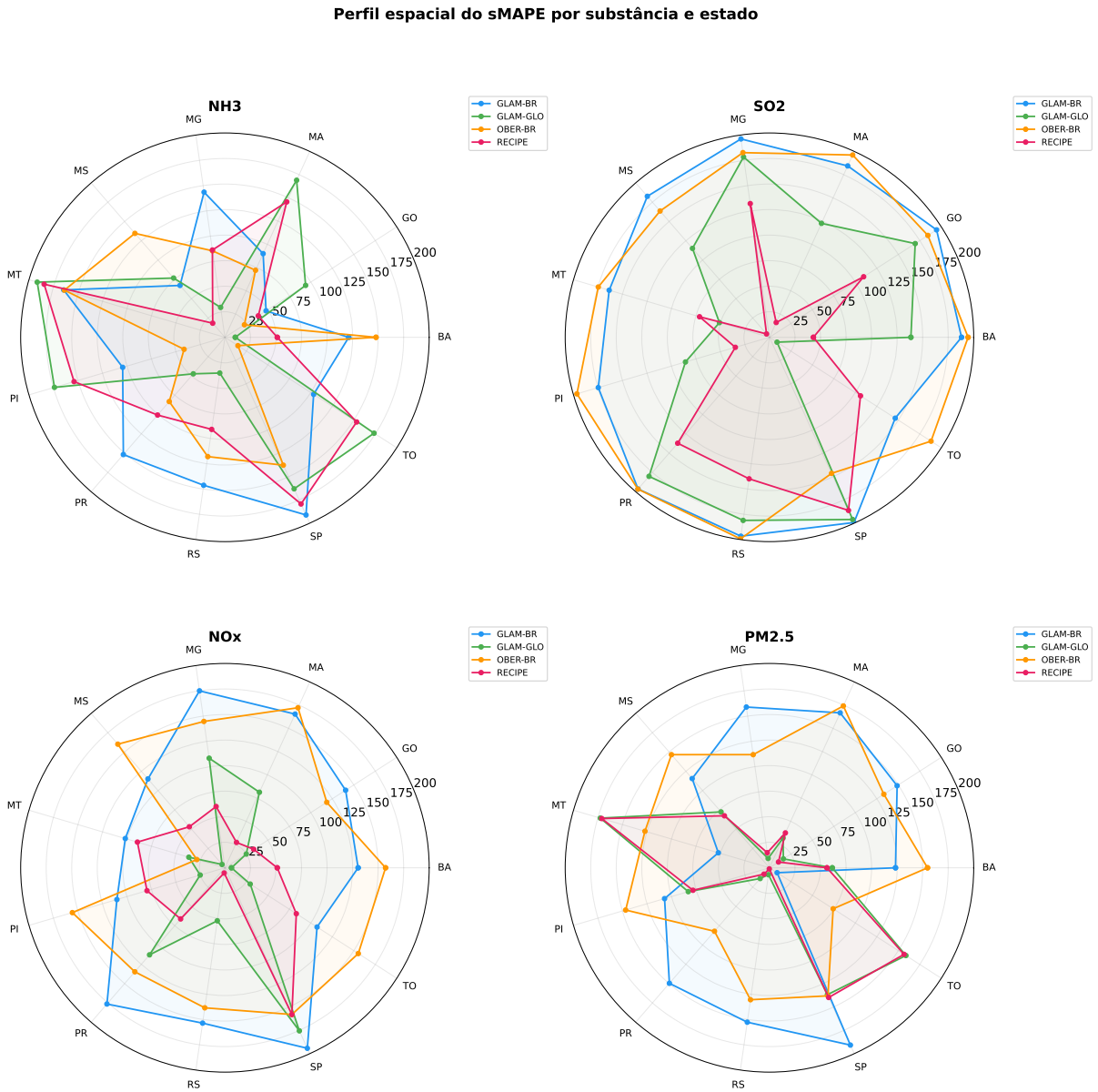

Figura 3: Perfil espacial do sMAPE por substância e estado. Cada eixo representa um estado; a distância do centro indica o sMAPE (%).

C.2.4 Método mais próximo do RAICV-BR por contexto

Das 44 combinações possíveis (11 estados × 4 substâncias), analisou-se qual método alternativo apresenta menor sMAPE em cada caso (Tabela 14 e Figura 4).

Tabela 14: Contagem de "vitórias" — método mais próximo do RAICV-BR

Método	**Vitórias (de 44)**	**Percentual**
ReCiPe	19	43%
GLAM-GLO	14	32%
OBER-BR	9	20%
GLAM-BR	2	5%

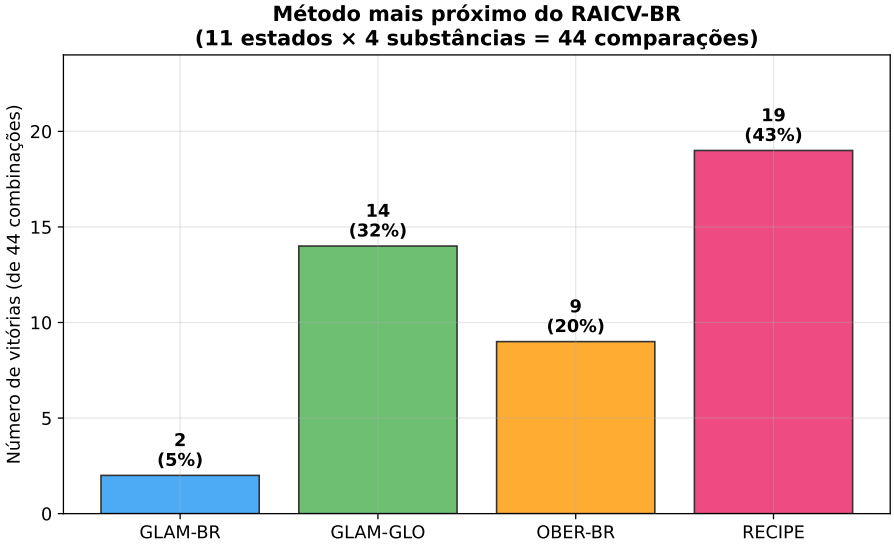

Figura 4: Número de combinações (estado × substância) em que cada método apresenta menor sMAPE em relação ao RAICV-BR.

O ReCiPe e o GLAM-GLO são, juntos, os mais próximos do RAICV-BR em 75% dos casos. Contudo, nenhum método isolado é consistentemente superior — a melhor alternativa depende tanto da substância quanto do estado em análise (Figura 5).

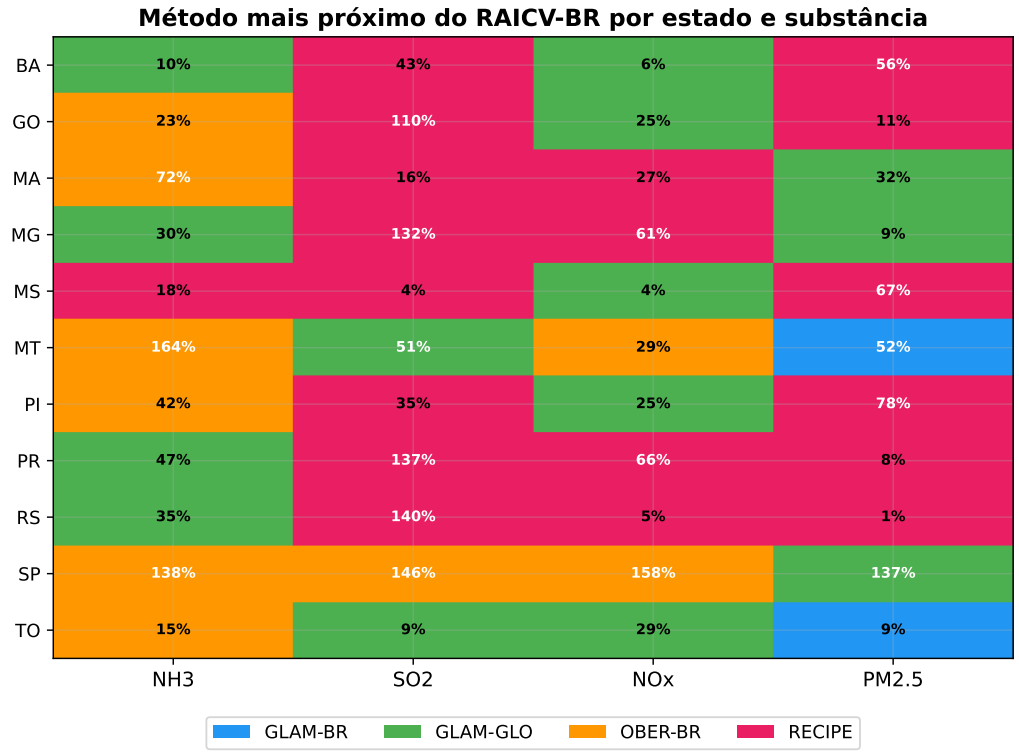

Figura 5: Método mais próximo do RAICV-BR por estado e substância, com sMAPE anotado em cada célula.

C.2.5 sMAPE médio por método e substância

A Figura 6 sumariza o sMAPE médio por método, desagregado por substância.

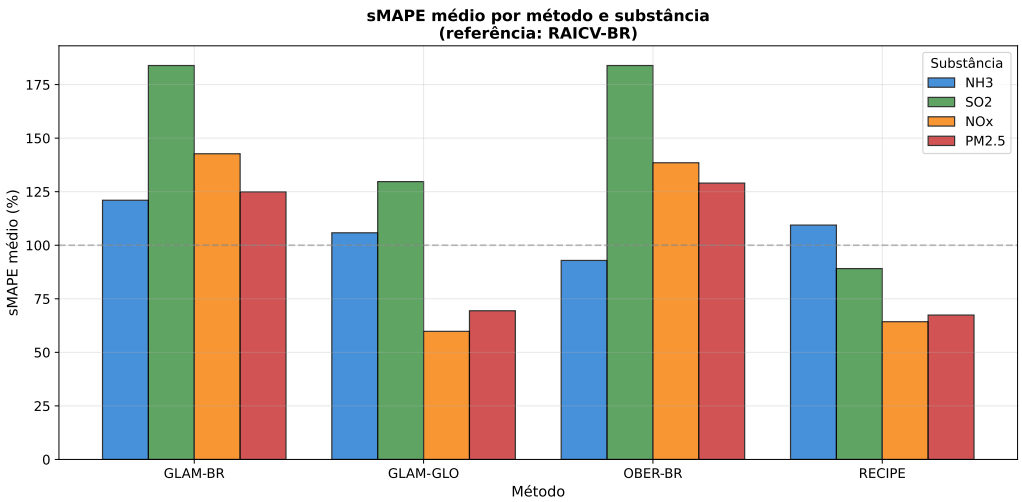

Figura 6: sMAPE médio por método e substância. A linha tracejada marca sMAPE = 100%.

C.3 Discussão

Os resultados desta comparação permitem destacar cinco pontos:

1. **A escolha do método de caracterização importa.** Para 3 de 4 substâncias, as diferenças entre métodos são estatisticamente significativas (Kruskal-Wallis, $p < 0{,}05$). Essa sensibilidade reforça a relevância de métodos regionalizados como o RAICV-BR.

2. **Regionalização não implica concordância.** Métodos regionalizados para o Brasil (GLAM-BR, Oberschelp) não são necessariamente mais próximos do RAICV-BR do que métodos globais (ReCiPe, GLAM-GLO). Cada modelo parte de premissas atmosféricas, resoluções espaciais e funções dose-resposta distintas.

3. **São Paulo é o caso mais crítico.** Todos os métodos alternativos divergem fortemente do RAICV-BR para SP (sMAPE médio = 168%), evidenciando que regiões com alta complexidade urbano-industrial requerem modelagem atmosférica dedicada.

4. **Nenhum método substitui todos os outros.** O ReCiPe é a melhor aproximação ao RAICV-BR em 43% dos casos, mas em 57% outro método é mais adequado. Isso demonstra que fatores de caracterização contextualizados são fundamentais para estudos de ACV que pretendam refletir as condições ambientais do Brasil.

5. **A ferramenta `olca-cf-converter` viabiliza comparações sistemáticas.** A capacidade de converter rapidamente diferentes conjuntos de FCs em pacotes importáveis pelo OpenLCA permitiu que esta comparação multimétodo fosse realizada de forma reprodutível e eficiente.